2019 International Conference on Robotics and Automation (ICRA)
Palais des congres de Montreal, Montreal, Canada, May 20-24, 2019

Neural Lander: Stable Drone Landing Control Using Learned Dynamics

Guanya Shi*,1, Xichen Shi*,1, Michael O'Connell*,1, Rose Yu[2], Kamyar Azizzadenesheli[3], Animashree Anandkumar[1], Yisong Yue[1], and Soon-Jo Chung[1]

***Abstract*— Precise near-ground trajectory control is difficult for multi-rotor drones, due to the complex aerodynamic effects caused by interactions between multi-rotor airflow and the environment. Conventional control methods often fail to properly account for these complex effects and fall short in accomplishing smooth landing. In this paper, we present a novel deep-learning-based robust nonlinear controller (*Neural-Lander*) that improves control performance of a quadrotor during landing. Our approach combines a nominal dynamics model with a Deep Neural Network (DNN) that learns high-order interactions. We apply spectral normalization (SN) to constrain the Lipschitz constant of the DNN. Leveraging this Lipschitz property, we design a nonlinear feedback linearization controller using the learned model and prove system stability with disturbance rejection. To the best of our knowledge, this is the first DNN-based nonlinear feedback controller with stability guarantees that can utilize arbitrarily large neural nets. Experimental results demonstrate that the proposed controller significantly outperforms a Baseline Nonlinear Tracking Controller in both landing and cross-table trajectory tracking cases. We also empirically show that the DNN generalizes well to unseen data outside the training domain.**

I. Introduction

Unmanned Aerial Vehicles (UAVs) require high precision control of aircraft positions, especially during landing and take-off. This problem is challenging largely due to complex interactions of rotor and wing airflows with the ground. The aerospace community has long identified such ground effect that can cause an increased lift force and a reduced aerodynamic drag. These effects can be both helpful and disruptive in flight stability [1], and the complications are exacerbated with multiple rotors. Therefore, performing automatic landing of UAVs is risk-prone, and requires expensive high-precision sensors as well as carefully designed controllers.

Compensating for ground effect is a long-standing problem in the aerial robotics community. Prior work has largely focused on mathematical modeling (e.g. [2]) as part of system identification (ID). These models are later used to approximate aerodynamics forces during flights close to the ground and combined with controller design for feed-forward cancellation (e.g. [3]). However, existing theoretical ground effect models are derived based on steady-flow conditions, whereas most practical cases exhibit unsteady flow. Alternative approaches, such as integral or adaptive control methods, often suffer from slow response and delayed feedback. [4] employs Bayesian Optimization for open-air control but not for take-off/landing. Given these limitations, the precision of existing fully automated systems for UAVs are still insufficient for landing and take-off, thereby necessitating the guidance of a human UAV operator during those phases.

To capture complex aerodynamic interactions without overly-constrained by conventional modeling assumptions, we take a machine-learning (ML) approach to build a black-box ground effect model using Deep Neural Networks (DNNs). However, incorporating such models into a UAV controller faces three key challenges. First, it is challenging to collect sufficient real-world training data, as DNNs are notoriously data-hungry. Second, due to high-dimensionality, DNNs can be unstable and generate unpredictable output, which makes the system susceptible to instability in the feedback control loop. Third, DNNs are often difficult to analyze, which makes it difficult to design provably stable DNN-based controllers.

The aforementioned challenges pervade previous works using DNNs to capture high-order non-stationary dynamics. For example, [5], [6] use DNNs to improve system ID of helicopter aerodynamics, but not for controller design. Other approaches aim to generate reference inputs or trajectories from DNNs [7]–[10]. However, these approaches can lead to challenging optimization problems [7], or heavily rely on well-designed closed-loop controller and require a large number of labeled training data [8]–[10]. A more classical approach of using DNNs is direct inverse control [11]–[13] but the non-parametric nature of a DNN controller also makes it challenging to guarantee stability and robustness to noise. [14] proposes a provably stable model-based Reinforcement Learning method based on Lyapunov analysis, but it requires a potentially expensive discretization step and relies on the native Lipschitz constant of the DNN.

Contributions. In this paper, we propose a learning-based controller, *Neural-Lander*, to improve the precision of quadrotor landing with guaranteed stability. Our approach directly learns the ground effect on coupled unsteady aerodynamics and vehicular dynamics. We use deep learning for system ID of residual dynamics and then integrate it with nonlinear feedback linearization control.

We train DNNs with layer-wise spectrally normalized weight matrices. We prove that the resulting controller is globally exponentially stable under bounded learning errors. This is achieved by exploiting the Lipschitz bound of spectrally normalized DNNs. It has earlier been shown that spectral normalization of DNNs leads to good generalization, i.e. stability in a learning-theoretic sense [15]. It is intriguing that spectral normalization simultaneously guarantees stability both in a learning-theoretic and a control-theoretic sense.

* These authors contributed equally to this work. [1]California Institute of Technology, [2]Northeastern University, [3]University of California, Irvine.

978-1-5386-6027-0/19/$31.00 ©2019 IEEE

Authorized licensed use limited to: CINVESTAV. Downloaded on September 11,2026 at 19:56:50 UTC from IEEE Xplore. Restrictions apply.

We evaluate *Neural-Lander* on trajectory tracking of quadrotor during take-off, landing and cross-table maneuvers. *Neural-Lander* is able to land a quadrotor much more accurately than a Baseline Nonlinear Tracking Controller with a pre-identified system. In particular, we show that compared to the baseline, *Neural-Lander* can decrease error in z axis from $0.13\,\mathrm{m}$ to 0, mitigate x and y drifts by as much as 90%, in the landing case. Meanwhile, *Neural-Lander* can decrease z error from $0.153\,\mathrm{m}$ to $0.027\,\mathrm{m}$, in the cross-table trajectory tracking task.[1] We also demonstrate that the learned model can handle temporal dependency, and is an improvement over the steady-state theoretical models.

II. Problem Statement: Quadrotor Landing

Given quadrotor states as global position $\mathbf{p} \in \mathbb{R}^3$, velocity $\mathbf{v} \in \mathbb{R}^3$, attitude rotation matrix $R \in \mathrm{SO}(3)$, and body angular velocity $\boldsymbol{\omega} \in \mathbb{R}^3$, we consider the following dynamics:

$$\dot{\mathbf{p}} = \mathbf{v}, \qquad m\dot{\mathbf{v}} = m\mathbf{g} + R\mathbf{f}_u + \mathbf{f}_a, \tag{1a}$$

$$\dot{R} = RS(\boldsymbol{\omega}), \qquad J\dot{\boldsymbol{\omega}} = J\boldsymbol{\omega} \times \boldsymbol{\omega} + \boldsymbol{\tau}_u + \boldsymbol{\tau}_a, \tag{1b}$$

where m and J are mass and inertia matrix of the system respectively, $S(\cdot)$ is skew-symmetric mapping. $\mathbf{g} = [0, 0, -g]^\top$ is the gravity vector, $\mathbf{f}_u = [0, 0, T]^\top$ and $\boldsymbol{\tau}_u = [\tau_x, \tau_y, \tau_z]^\top$ are the total thrust and body torques from four rotors predicted by a nominal model. We use $\boldsymbol{\eta} = [T, \tau_x, \tau_y, \tau_z]^\top$ to denote the output wrench. Typical quadrotor control input uses squared motor speeds $\mathbf{u} = [n_1^2, n_2^2, n_3^2, n_4^2]^\top$, and is linearly related to the output wrench $\boldsymbol{\eta} = B_0\mathbf{u}$, with

$$B_0 = \begin{bmatrix} c_T & c_T & c_T & c_T \\ 0 & c_T l_{\mathrm{arm}} & 0 & -c_T l_{\mathrm{arm}} \\ -c_T l_{\mathrm{arm}} & 0 & c_T l_{\mathrm{arm}} & 0 \\ -c_Q & c_Q & -c_Q & c_Q \end{bmatrix}, \tag{2}$$

where c_T and c_Q are rotor force and torque coefficients, and l_{arm} denotes the length of rotor arm. The key difficulty of precise landing is the influence of unknown disturbance forces $\mathbf{f}_a = [f_{a,x}, f_{a,y}, f_{a,z}]^\top$ and torques $\boldsymbol{\tau}_a = [\tau_{a,x}, \tau_{a,y}, \tau_{a,z}]^\top$, which originate from complex aerodynamic interactions between the quadrotor and the environment.

Problem Statement: We aim to improve controller accuracy by learning the unknown disturbance forces $\mathbf{f}_a$ and torques $\boldsymbol{\tau}_a$ in (1). As we mainly focus on landing and take-off tasks, the attitude dynamics is limited and the aerodynamic disturbance torque $\boldsymbol{\tau}_a$ is bounded. Thus position dynamics (1a) and $\mathbf{f}_a$ will our primary concern. We first approximate $\mathbf{f}_a$ using a DNN with spectral normalization to guarantee its Lipschitz constant, and then incorporate the DNN in our exponentially-stabilizing controller. Training is done off-line, and the learned dynamics is applied in the on-board controller in real-time to achieve smooth landing and take-off.

III. Dynamics Learning using DNN

We learn the unknown disturbance force $\mathbf{f}_a$ using a DNN with Rectified Linear Units (ReLU) activation. In general, DNNs equipped with ReLU converge faster during training, demonstrate more robust behavior with respect to changes in hyperparameters, and have fewer vanishing gradient problems compared to other activation functions such as *sigmoid* [16].

[1]Demo videos: `https://youtu.be/FLLsG0S78ik`

A. ReLU Deep Neural Networks

A ReLU deep neural network represents the functional mapping from the input $\mathbf{x}$ to the output $f(\mathbf{x}, \boldsymbol{\theta})$, parameterized by the DNN weights $\boldsymbol{\theta} = W^1, \cdots, W^{L+1}$:

$$f(\mathbf{x}, \boldsymbol{\theta}) = W^{L+1}\phi(W^L(\phi(W^{L-1}(\cdots\phi(W^1\mathbf{x})\cdots)))), \tag{3}$$

where the activation function $\phi(\cdot) = \max(\cdot, 0)$ is called the element-wise ReLU function. ReLU is less computationally expensive than *tanh* and *sigmoid* because it involves simpler mathematical operations. However, deep neural networks are usually trained by first-order gradient based optimization, which is highly sensitive on the curvature of the training objective and can be unstable [17]. To alleviate this issue, we apply the spectral normalization technique [15].

B. Spectral Normalization

Spectral normalization stabilizes DNN training by constraining the Lipschitz constant of the objective function. Spectrally normalized DNNs have also been shown to generalize well [18], which is an indication of stability in machine learning. Mathematically, the Lipschitz constant of a function $\|f\|_{\mathrm{Lip}}$ is defined as the smallest value such that

$$\forall\, \mathbf{x}, \mathbf{x}' :\ \|f(\mathbf{x}) - f(\mathbf{x}')\|_2 / \|\mathbf{x} - \mathbf{x}'\|_2 \le \|f\|_{\mathrm{Lip}}.$$

It is known that the Lipschitz constant of a general differentiable function f is the maximum spectral norm (maximum singular value) of its gradient over its domain $\|f\|_{\mathrm{Lip}} = \sup_{\mathbf{x}} \sigma(\nabla f(\mathbf{x}))$.

The ReLU DNN in (3) is a composition of functions. Thus we can bound the Lipschitz constant of the network by constraining the spectral norm of each layer $g^l(\mathbf{x}) = \phi(W^l\mathbf{x})$. Therefore, for a linear map $g(\mathbf{x}) = W\mathbf{x}$, the spectral norm of each layer is given by $\|g\|_{\mathrm{Lip}} = \sup_{\mathbf{x}} \sigma(\nabla g(\mathbf{x})) = \sup_{\mathbf{x}} \sigma(W) = \sigma(W)$. Using the fact that the Lipschitz norm of ReLU activation function $\phi(\cdot)$ is equal to 1, with the inequality $\|g_1 \circ g_2\|_{\mathrm{Lip}} \le \|g_1\|_{\mathrm{Lip}} \cdot \|g_2\|_{\mathrm{Lip}}$, we can find the following bound on $\|f\|_{\mathrm{Lip}}$:

$$\|f\|_{\mathrm{Lip}} \le \|g^{L+1}\|_{\mathrm{Lip}} \cdot \|\phi\|_{\mathrm{Lip}} \cdots \|g^1\|_{\mathrm{Lip}} = \prod_{l=1}^{L+1} \sigma(W^l). \tag{4}$$

In practice, we can apply spectral normalization to the weight matrices in each layer during training as follows:

$$\bar{W} = W/\sigma(W) \cdot \gamma^{\frac{1}{L+1}}, \tag{5}$$

where γ is the intended Lipschitz constant for the DNN. The following lemma bounds the Lipschitz constant of a ReLU DNN with spectral normalization.

Lemma 3.1: For a multi-layer ReLU network $f(\mathbf{x}, \boldsymbol{\theta})$, defined in (3) without an activation function on the output layer. Using spectral normalization, the Lipschitz constant of the entire network satisfies:

$$\|f(\mathbf{x}, \bar{\boldsymbol{\theta}})\|_{\mathrm{Lip}} \le \gamma,$$

with spectrally-normalized parameters $\bar{\boldsymbol{\theta}} = \bar{W}^1, \cdots, \bar{W}^{L+1}$.

Proof: As in (4), the Lipschitz constant can be written as a composition of spectral norms over all layers. The proof follows from the spectral norms constrained as in (5). ■

Authorized licensed use limited to: CINVESTAV. Downloaded on September 11,2026 at 19:56:50 UTC from IEEE Xplore. Restrictions apply.

C. Constrained Training

We apply gradient-based optimization to train the ReLU DNN with a bounded Lipschitz constant. Estimating $\mathbf{f}_a$ in (1) boils down to optimizing the parameters $\boldsymbol{\theta}$ in the ReLU network in (3), given the observed value of $\mathbf{x}$ and the target output. In particular, we want to control the Lipschitz constant of the ReLU network.

The optimization objective is as follows, where we minimize the prediction error with constrained Lipschitz constant:

$$\begin{aligned} \underset{\boldsymbol{\theta}}{\text{minimize}} \quad & \sum_{t=1}^{T} \frac{1}{T} \|\mathbf{y}_t - f(\mathbf{x}_t, \boldsymbol{\theta})\|_2 \\ \text{subject to} \quad & \|f\|_{\text{Lip}} \leq \gamma. \end{aligned} \tag{6}$$

In our case, $\mathbf{y}_t$ is the observed disturbance forces and $\mathbf{x}_t$ is the observed states and control inputs. According to the upper bound in (4), we can substitute the constraint by minimizing the spectral norm of the weights in each layer. We use stochastic gradient descent (SGD) to optimize (6) and apply spectral normalization to regulate the weights. From Lemma 3.1, the trained ReLU DNN has a Lipschitz constant.

IV. Neural Lander Controller Design

Our *Neural-Lander* controller for 3-D trajectory tracking is constructed as a nonlinear feedback linearization controller whose stability guarantees are obtained using the spectral normalizaion of the DNN-based ground-effect model. We then exploit the Lipschitz property of the DNN to solve for the resulting control input using fixed-point iteration.

A. Reference Trajectory Tracking

The position tracking error is defined as $\tilde{\mathbf{p}} = \mathbf{p} - \mathbf{p}_d$. Our controller uses a composite variable $\mathbf{s} = 0$ as a manifold on which $\tilde{\mathbf{p}}(t) \to 0$ exponentially:

$$\mathbf{s} = \dot{\tilde{\mathbf{p}}} + \Lambda \tilde{\mathbf{p}} = \dot{\mathbf{p}} - \mathbf{v}_r \tag{7}$$

with Λ as a positive definite or diagonal matrix. Now the trajectory tracking problem is transformed to tracking a reference velocity $\mathbf{v}_r = \dot{\mathbf{p}}_d - \Lambda \tilde{\mathbf{p}}$.

Using the methods described in Sec. III, we define $\hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u})$ as the DNN approximation to the disturbance aerodynamic forces, with $\boldsymbol{\zeta}$ being the partial states used as input features to the network. We design the total desired rotor force $\mathbf{f}_d$ as

$$\mathbf{f}_d = (R\mathbf{f}_u)_d = \bar{\mathbf{f}}_d - \hat{\mathbf{f}}_a, \text{with} \;\; \bar{\mathbf{f}}_d = m\dot{\mathbf{v}}_r - K_v \mathbf{s} - m\mathbf{g}. \tag{8}$$

Substituting (8) into (1), the closed-loop dynamics would simply become $m\dot{\mathbf{s}} + K_v \mathbf{s} = \boldsymbol{\epsilon}$, with approximation error $\boldsymbol{\epsilon} = \mathbf{f}_a - \hat{\mathbf{f}}_a$. Hence, $\tilde{\mathbf{p}}(t) \to \mathbf{0}$ globally and exponentially with bounded error, as long as $\|\boldsymbol{\epsilon}\|$ is bounded [19]–[21].

Consequently, desired total thrust T_d and desired force direction $\hat{k}_d$ can be computed as

$$T_d = \mathbf{f}_d \cdot \hat{k}, \;\text{ and }\; \hat{k}_d = \mathbf{f}_d / \left\|\mathbf{f}_d\right\|, \tag{9}$$

with $\hat{k}$ being the unit vector of rotor thrust direction (typically z-axis in quadrotors). Using $\hat{k}_d$ and fixing a desired yaw angle, desired attitude R_d can be deduced [22]. We assume that a nonlinear attitude controller uses the desired torque $\boldsymbol{\tau}_d$ from rotors to track $R_d(t)$. One such example is in [21]:

$$\boldsymbol{\tau}_d = J\dot{\boldsymbol{\omega}}_r - J\boldsymbol{\omega} \times \boldsymbol{\omega}_r - K_\omega(\boldsymbol{\omega} - \boldsymbol{\omega}_r), \tag{10}$$

where the reference angular rate $\boldsymbol{\omega}_r$ is designed similar to (7), so that when $\boldsymbol{\omega} \to \boldsymbol{\omega}_r$, exponential trajectory tracking of a desired attitude $R_d(t)$ is guaranteed within some bounded error in the presence of bounded disturbance torques.

B. Learning-based Discrete-time Nonlinear Controller

From (2), (9) and (10), we can relate the desired wrench $\boldsymbol{\eta}_d = [T_d, \boldsymbol{\tau}_d^\top]^\top$ with the control signal $\mathbf{u}$ through

$$B_0 \mathbf{u} = \boldsymbol{\eta}_d = \begin{bmatrix} \left(\bar{\mathbf{f}}_d - \hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u})\right) \cdot \hat{k} \\ \boldsymbol{\tau}_d \end{bmatrix}. \tag{11}$$

Because of the dependency of $\hat{\mathbf{f}}_a$ on $\mathbf{u}$, the control synthesis problem here is non-affine. Therefore, we propose the following fixed-point iteration method for solving (11):

$$\mathbf{u}_k = B_0^{-1} \boldsymbol{\eta}_d \left(\mathbf{u}_{k-1}\right), \tag{12}$$

where $\mathbf{u}_k$ and $\mathbf{u}_{k-1}$ are the control input for current and previous time-step in the discrete-time controller. Next, we prove the stability of the system and convergence of the control inputs in (12).

V. Nonlinear Stability Analysis

The closed-loop tracking error analysis provides a direct correlation on how to tune the neural network and controller parameter to improve control performance and robustness.

A. Control Allocation as Contraction Mapping

We first show that the control input $\mathbf{u}_k$ converges to the solution of (11) when all states are fixed.

Lemma 5.1: Define mapping $\mathbf{u}_k = \mathcal{F}(\mathbf{u}_{k-1})$ based on (12) and fix all current states:

$$\mathcal{F}(\mathbf{u}) = B_0^{-1} \begin{bmatrix} \left(\bar{\mathbf{f}}_d - \hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u})\right) \cdot \hat{k} \\ \boldsymbol{\tau}_d \end{bmatrix}. \tag{13}$$

If $\hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u})$ is L_a-Lipschitz continuous, and $\sigma(B_0^{-1}) \cdot L_a < 1$; then $\mathcal{F}(\cdot)$ is a contraction mapping, and $\mathbf{u}_k$ converges to unique solution of $\mathbf{u}^* = \mathcal{F}(\mathbf{u}^*)$.

Proof: $\forall\, \mathbf{u}_1, \mathbf{u}_2 \in \mathcal{U}$ with $\mathcal{U}$ being a compact set of feasible control inputs; and given fixed states as $\bar{\mathbf{f}}_d$, $\boldsymbol{\tau}_d$ and $\hat{k}$, then:

$$\begin{aligned} \|\mathcal{F}(\mathbf{u}_1) - \mathcal{F}(\mathbf{u}_2)\|_2 &= \left\| B_0^{-1} \left(\hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \boldsymbol{u}_1) - \hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \boldsymbol{u}_2)\right) \right\|_2 \\ &\leq \sigma(B_0^{-1}) \cdot L_a \left\|\mathbf{u}_1 - \mathbf{u}_2\right\|_2. \end{aligned}$$

Thus, $\exists\; \alpha < 1, \text{s.t}\; \|\mathcal{F}(\mathbf{u}_1) - \mathcal{F}(\mathbf{u}_2)\|_2 < \alpha \left\|\mathbf{u}_1 - \mathbf{u}_2\right\|_2$. Hence, $\mathcal{F}(\cdot)$ is a contraction mapping. ■

Authorized licensed use limited to: CINVESTAV. Downloaded on September 11,2026 at 19:56:50 UTC from IEEE Xplore. Restrictions apply.

B. Stability of Learning-based Nonlinear Controller

Before continuing to prove the stability of the full system, we make the following assumptions.

Assumption 1: The desired states along the position trajectory $\mathbf{p}_d(t)$, $\dot{\mathbf{p}}_d(t)$, and $\ddot{\mathbf{p}}_d(t)$ are bounded.

Assumption 2: One-step difference of control signal satisfies $\|\mathbf{u}_k - \mathbf{u}_{k-1}\| \leq \rho \|\mathbf{s}\|$ with a small positive ρ.

Here we provide the intuition behind this assumption. From (13), we can derive the following approximate relation with $\Delta(\cdot)_k = \|(\cdot)_k - (\cdot)_{k-1}\|$:

$$\Delta u_k \leq \sigma(B_0^{-1})\big(L_a \Delta u_{k-1} + L_a \Delta \zeta_k + m\Delta \dot{v}_{r,k} + \lambda_{\max}(K_v)\Delta s_k + \Delta \tau_{d,k}\big).$$

Because update rate of attitude controller ($> 100\,\text{Hz}$) and motor speed control ($> 5\,\text{kHz}$) are much higher than that of the position controller ($\approx 10\,\text{Hz}$), in practice, we can safely neglect Δs_k, $\Delta \dot{v}_{r,k}$, and $\Delta \zeta_k$ in one update (Theorem 11.1 [23]). Furthermore, $\Delta \tau_{d,k}$ can be limited internally by the attitude controller. It leads to:

$$\Delta u_k \leq \sigma(B_0^{-1})\big(L_a \Delta u_{k-1} + c\big),$$

with c being a small constant and $\sigma(B_0^{-1}) \cdot L_a < 1$ from Lemma. 5.1, we can deduce that Δu rapidly converges to a small ultimate bound between each position controller update.

Assumption 3: The learning error of $\hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u})$ over the compact sets $\boldsymbol{\zeta} \in \mathcal{Z}$, $\mathbf{u} \in \mathcal{U}$ is upper bounded by $\epsilon_m = \sup_{\boldsymbol{\zeta}\in\mathcal{Z}, \mathbf{u}\in\mathcal{U}} \|\boldsymbol{\epsilon}(\boldsymbol{\zeta}, \mathbf{u})\|$, where $\boldsymbol{\epsilon}(\boldsymbol{\zeta}, \mathbf{u}) = \mathbf{f}_a(\boldsymbol{\zeta}, \mathbf{u}) - \hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u})$.

DNNs have been shown to generalize well to the set of unseen events that are from almost the same distribution as training set [24], [25]. This empirical observation is also theoretically studied in order to shed more light toward an understanding of the complexity of these models [18], [26]–[28]. Based on the above assumptions, we can now present our overall stability and robustness result.

Theorem 5.2: Under Assumptions 1-3, for a time-varying $\mathbf{p}_d(t)$, the controller defined in (8) and (12) with $\lambda_{\min}(K_v) > L_a\rho$ achieves exponential convergence of composite variable $\mathbf{s}$ to error ball $\lim_{t\to\infty} \|\mathbf{s}(t)\| = \epsilon_m / (\lambda_{\min}(K_v) - L_a\rho)$ with rate $((\lambda_{\min}(K_v) - L_a\rho)/m$. And $\tilde{\mathbf{p}}$ exponentially converges to error ball

$$\lim_{t\to\infty} \|\tilde{\mathbf{p}}(t)\| = \frac{\epsilon_m}{\lambda_{\min}(\Lambda)(\lambda_{\min}(K_v) - L_a\rho)} \tag{14}$$

with rate $\lambda_{\min}(\Lambda)$.

Proof: We begin the proof by selecting a Lyapunov function as $\mathcal{V}(\mathbf{s}) = \frac{1}{2}m\|\mathbf{s}\|^2$, then by applying the controller (8), we get the time-derivative of $\mathcal{V}$:

$$\begin{aligned}\dot{\mathcal{V}} &= \mathbf{s}^\top\big(-K_v\mathbf{s} + \hat{\mathbf{f}}_a(\boldsymbol{\zeta}_k, \mathbf{u}_k) - \hat{\mathbf{f}}_a(\boldsymbol{\zeta}_k, \mathbf{u}_{k-1}) + \boldsymbol{\epsilon}(\boldsymbol{\zeta}_k, \mathbf{u}_k)\big) \\ &\leq -\mathbf{s}^\top K_v \mathbf{s} + \|\mathbf{s}\|(\|\hat{\mathbf{f}}_a(\boldsymbol{\zeta}_k, \mathbf{u}_k) - \hat{\mathbf{f}}_a(\boldsymbol{\zeta}_k, \mathbf{u}_{k-1})\| + \epsilon_m)\end{aligned}$$

Let $\lambda = \lambda_{\min}(K_v)$ denote the minimum eigenvalue of the positive-definite matrix K_v. By applying the Lipschitz property of $\hat{\mathbf{f}}_a$ theorem 3.1 and Assumption 2, we obtain

$$\dot{\mathcal{V}} \leq -\frac{2\,(\lambda - L_a\rho)}{m}\mathcal{V} + \sqrt{\frac{2\mathcal{V}}{m}}\epsilon_m$$

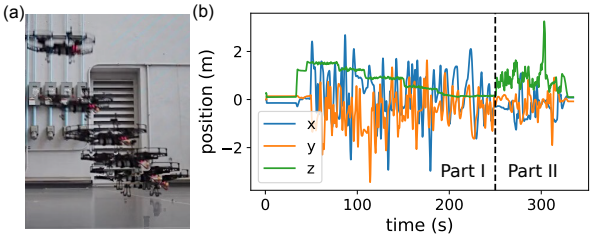


Fig. 1: (a) Intel Aero drone; (b) Training data trajectory. Part I (0 to 250 s) contains maneuvers at different heights (0.05 m to 1.50 m). Part II (250 s to 350 s) includes random x, y, and z motions for maximum state-space coverage.

Using the Comparison Lemma [23], we define $\mathcal{W}(t) = \sqrt{\mathcal{V}(t)} = \sqrt{m/2}\|\mathbf{s}\|$ and $\dot{\mathcal{W}} = \dot{\mathcal{V}} / \left(2\sqrt{\mathcal{V}}\right)$ to obtain

$$\|\mathbf{s}(t)\| \leq \|\mathbf{s}(t_0)\| \exp\left(-\frac{\lambda - L_a\rho}{m}(t - t_0)\right) + \frac{\epsilon_m}{\lambda - L_a\rho}$$

It can be shown that this leads to finite-gain $\mathcal{L}_p$ stability and input-to-state stability (ISS) [29]. Furthermore, the hierarchical combination between $\mathbf{s}$ and $\tilde{\mathbf{p}}$ in (7) results in $\lim_{t\to\infty} \|\tilde{\mathbf{p}}(t)\| = \lim_{t\to\infty} \|\mathbf{s}(t)\| / \lambda_{\min}(\Lambda)$, yielding (14). ■

VI. Experiments

In our experiments, we evaluate both the generalization performance of our DNN as well as the overall control performance of *Neural-Lander*. The experimental setup is composed of a motion capture system with 17 cameras, a WiFi router for communication, and an Intel Aero drone, weighing 1.47 kg with an onboard Linux computer (2.56 GHz Intel Atom x7 processor, 4 GB DDR3 RAM). We retrofitted the drone with eight reflective infrared markers for accurate position, attitude and velocity estimation at 100Hz. The Intel Aero drone and the test space are shown in Fig. 1(a).

A. Bench Test

To identify a good nominal model, we first measured the mass, m, diameter of the rotor, D, the air density, ρ, gravity, g. Then we performed bench test to determine the thrust constant, c_T, as well as the non-dimensional thrust coefficient $C_T = \frac{c_T}{\rho D^4}$. Note that C_T is a function of propeller speed n, and here we picked a nominal value at $n = 2000\,\text{RPM}$.

B. Real-World Flying Data and Preprocessing

To estimate the disturbance force $\mathbf{f}_a$, an expert pilot manually flew the drone at different heights, and we collected training data consisting of sequences of state estimates and control inputs $\{(\mathbf{p}, \mathbf{v}, R, \mathbf{u}), \mathbf{y}\}$ where $\mathbf{y}$ is the observed value of $\mathbf{f}_a$. We utilized the relation $\mathbf{f}_a = m\dot{\mathbf{v}} - m\mathbf{g} - R\mathbf{f}_u$ from (1) to calculate $\mathbf{f}_a$, where $\mathbf{f}_u$ is calculated based on the nominal c_T from the bench test in Sec. VI-A. Our training set is a single continuous trajectory with varying heights and velocities. The trajectory has two parts shown in Fig. 1(b). We aim to learn the ground effect through Part I of the training set, and other aerodynamics forces such as air drag through Part II.

Authorized licensed use limited to: CINVESTAV. Downloaded on September 11,2026 at 19:56:50 UTC from IEEE Xplore. Restrictions apply.

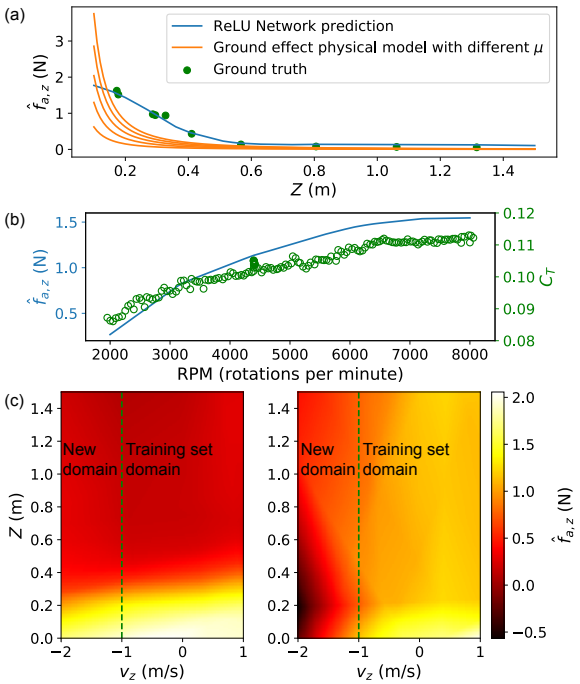


Fig. 2: (a) Learned $\hat{f}_{a,z}$ compared to the ground effect model with respect to height z, with $v_z = v_x = v_y = 0\,\mathrm{m/s}$, $R = I$, $\mathbf{u} = 6400\,\mathrm{RPM}$. Ground truth points are from hovering data at different heights. (b) Learned $\hat{f}_{a,z}$ with respect to rotation speed n ($z = 0.2\,\mathrm{m}$, $v_z = 0\,\mathrm{m/s}$), compared to C_T measured in the bench test. (c) Heatmaps of learned $\hat{f}_{a,z}$ versus z and v_z. (Left) ReLU network with spectral normalization. (Right) ReLU network without spectral normalization.

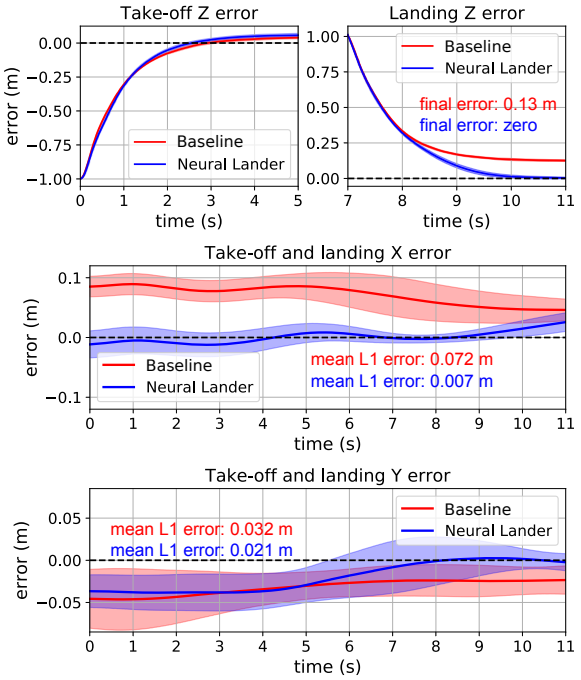


Fig. 3: Baseline Controller and *Neural-Lander* performance in take-off and landing. Means (solid curves) and standard deviations (shaded areas) of 10 trajectories.

C. DNN Prediction Performance

We train a deep ReLU network $\hat{\mathbf{f}}_a(\boldsymbol{\zeta}, \mathbf{u}) = \hat{\mathbf{f}}_a(z, \mathbf{v}, R, \mathbf{u})$, with z, $\mathbf{v}$, R, $\mathbf{u}$ corresponding to global height, global velocity, attitude, and control input. We build the ReLU network using PyTorch [30]. Our ReLU network consists of four fully-connected hidden layers, with input and the output dimensions 12 and 3, respectively. We use spectral normalization (5) to constrain the Lipschitz constant of the DNN.

We compare the near-ground estimation accuracy our DNN model with existing 1D steady ground effect model [1], [3]:

$$T(n,z) = \frac{n^2}{1-\mu(\frac{D}{8z})^2} c_T(n) = n^2 c_T(n_0) + \bar{f}_{a,z}, \qquad (15)$$

where T is the thrust generated by propellers, n is the rotation speed, n_0 is the idle RPM, and μ depends on the number and the arrangement of propellers ($\mu = 1$ for a single propeller, but must be tuned for multiple propellers). Note that c_T is a function of n. Thus, we can derive $\bar{f}_{a,z}(n,z)$ from $T(n,z)$.

Fig. 2(a) shows the comparison between the estimated $\mathbf{f}_a$ from DNN and the theoretical ground effect model (15) at different z (assuming $T = mg$ when $z = \infty$). We can see that our DNN can achieve much better estimates than the theoretical ground effect model. We further investigate the trend of $\bar{f}_{a,z}$ with respect to the rotation speed n. Fig. 2(b) shows the learned $\hat{f}_{a,z}$ over the rotation speed n at a given height, in comparison with the C_T measured from the bench test. We observe that the increasing trend of the estimates $\hat{f}_{a,z}$ is consistent with bench test results for C_T.

To understand the benefits of SN, we compared $\hat{f}_{a,z}$ predicted by the DNNs trained both with and without SN as shown in Fig. 2(c). Note that v_z from $-1\,\mathrm{m/s}$ to $1\,\mathrm{m/s}$ is covered in our training set, but $-2\,\mathrm{m/s}$ to $-1\,\mathrm{m/s}$ is not. We observe the following differences:

1) Ground effect: $\hat{f}_{a,z}$ increases as z decreases, which is also shown in Fig. 2(a).
2) Air drag: $\hat{f}_{a,z}$ increases as the drone goes down ($v_z < 0$) and it decreases as the drone goes up ($v_z > 0$).
3) Generalization: the spectral normalized DNN is much smoother and can also generalize to new input domains not contained in the training set.

In [18], the authors theoretically show that spectral normalization can provide tighter generalization guarantees on unseen data, which is consistent with our empirical observation. We will connect generalization theory more tightly with our robustness guarantees in the future.

D. Baseline Controller

We compared the *Neural-Lander* with a Baseline Nonlinear Tracking Controller. We implemented both a Baseline Controller similar to (7) and (8) with $\hat{\mathbf{f}}_a \equiv 0$, as well as an integral controller variation with $\mathbf{v}_r = \dot{\mathbf{p}}_d - 2\Lambda\tilde{\mathbf{p}} - \Lambda^2 \int_0^t \tilde{\mathbf{p}}(\tau)d\tau$. Though an integral gain can cancel steady-state error during set-point regulation, our flight results showed that the performance can be sensitive to the integral gain,

Authorized licensed use limited to: CINVESTAV. Downloaded on September 11,2026 at 19:56:50 UTC from IEEE Xplore. Restrictions apply.

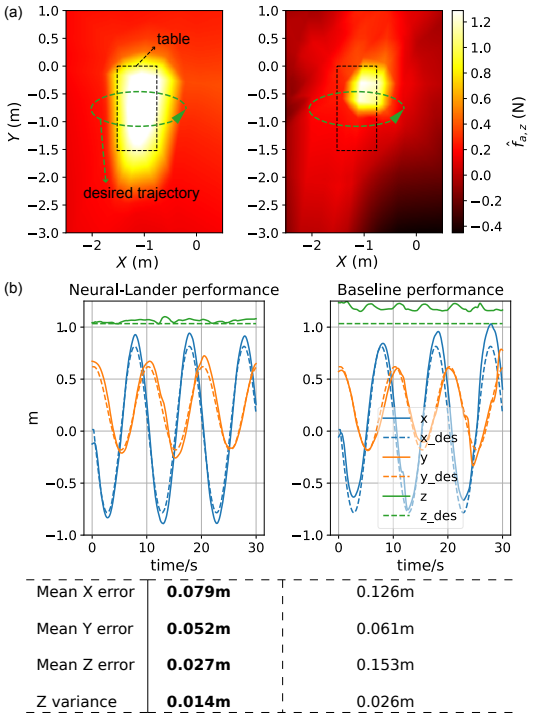


Fig. 4: (a) Heatmaps of learned $\hat{f}_{a,z}$ versus x and y, with other inputs fixed. (Left) ReLU network with spectral normalization. (Right) ReLU network without spectral normalization. (b) Tracking performance and statistics.

especially during trajectory tracking. This can be seen in the demo video.[2]

E. Setpoint Regulation Performance

First, we tested the two controllers' performance in take-off/landing, by commanding position setpoint $\mathbf{p}_d$, from $(0,0,0)$, to $(0,0,1)$, then back to $(0,0,0)$, with $\dot{\mathbf{p}}_d \equiv 0$. From Fig. 3, we can conclude that there are two main benefits of our *Neural-Lander*. (a) *Neural-Lander* can control the drone to precisely and smoothly land on the ground surface while the Baseline Controller struggles to achieve 0 terminal height due to the ground effect. (b) *Neural-Lander* can mitigate drifts in $x-y$ plane, as it also learned about additional aerodynamics such as air drag.

In experiments, we observed the *Neural-Lander* without spectral normalization can even result in unexpected controller outputs leading to crash, which empirically implies the necessity of SN in training the DNN and designing the controller.

F. Trajectory Tracking Performance

To show that our algorithm can handle more complicated environments where physics-based modelling of dynamics would be substantially more difficult, we devise a task of tracking an elliptic trajectory very close to a table with a period of 10 seconds shown in Fig. 4. The trajectory is partially over the table with significant ground effects, and a sharp transition to free space at the edge of the table. We compared the performance of both *Neural-Lander* and Baseline Controller on this test.

In order to model the complex dynamics near the table, we manually flew the drone in the space close to the table to collect another data set. We trained a new ReLU DNN model with x-y positions as additional input features: $\hat{\mathbf{f}}_a(\mathbf{p}, \mathbf{v}, R, \mathbf{u})$. Similar to the setpoint experiment, the benefit of spectral normalization can be seen in Fig. 4(a), where only the spectrally-normalized DNN exhibits a clear table boundary.

Fig. 4(b) shows that *Neural-Lander* outperformed the Baseline Controller for tracking the desired position trajectory in all x, y, and z axes. Additionally, *Neural-Lander* showed a lower variance in height, even at the edge of the table, as the controller captured the changes in ground effects when the drone flew over the table.

In summary, the experimental results with multiple ground interaction scenarios show that much smaller tracking errors are obtained by *Neural-Lander*, which is essentially the nonlinear tracking controller with feedforward cancellation of a spectrally-normalized DNN.

VII. Conclusions

In this paper, we present *Neural-Lander*, a deep learning based nonlinear controller with guaranteed stability for precise quadrotor landing. Compared to the Baseline Controller, *Neural-Lander* is able to significantly improve control performance. The main benefits are (1) our method can learn from coupled unsteady aerodynamics and vehicle dynamics to provide more accurate estimates than theoretical ground effect models, (2) our model can capture both the ground effect and other non-dominant aerodynamics and outperforms the conventional controller in all axes (x, y and z), and (3) we provide rigorous theoretical analysis of our method and guarantee the stability of the controller, which also implies generalization to unseen domains.

Future work includes further generalization of the capabilities of *Neural-Lander* handling unseen state and disturbance domains, such as those generated by a wind fan array.

Acknowledgement

The authors thank Joel Burdick, Mory Gharib and Daniel Pastor Moreno. The work is funded in part by Caltech's Center for Autonomous Systems and Technologies and Raytheon Company.

[2]Demo videos: `https://youtu.be/FLLsG0S78ik`

Authorized licensed use limited to: CINVESTAV. Downloaded on September 11,2026 at 19:56:50 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] I. Cheeseman and W. Bennett, "The effect of ground on a helicopter rotor in forward flight," 1955.

[2] K. Nonaka and H. Sugizaki, "Integral sliding mode altitude control for a small model helicopter with ground effect compensation," in *American Control Conference (ACC), 2011*. IEEE, 2011, pp. 202–207.

[3] L. Danjun, Z. Yan, S. Zongying, and L. Geng, "Autonomous landing of quadrotor based on ground effect modelling," in *Control Conference (CCC), 2015 34th Chinese*. IEEE, 2015, pp. 5647–5652.

[4] F. Berkenkamp, A. P. Schoellig, and A. Krause, "Safe controller optimization for quadrotors with Gaussian processes," in *Proc. of the IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 493–496. [Online]. Available: https://arxiv.org/abs/1509.01066

[5] P. Abbeel, A. Coates, and A. Y. Ng, "Autonomous helicopter aerobatics through apprenticeship learning," *The International Journal of Robotics Research*, vol. 29, no. 13, pp. 1608–1639, 2010.

[6] A. Punjani and P. Abbeel, "Deep learning helicopter dynamics models," in *Robotics and Automation (ICRA), 2015 IEEE International Conference on*. IEEE, 2015, pp. 3223–3230.

[7] S. Bansal, A. K. Akametalu, F. J. Jiang, F. Laine, and C. J. Tomlin, "Learning quadrotor dynamics using neural network for flight control," in *Decision and Control (CDC), 2016 IEEE 55th Conference on*. IEEE, 2016, pp. 4653–4660.

[8] Q. Li, J. Qian, Z. Zhu, X. Bao, M. K. Helwa, and A. P. Schoellig, "Deep neural networks for improved, impromptu trajectory tracking of quadrotors," in *Robotics and Automation (ICRA), 2017 IEEE International Conference on*. IEEE, 2017, pp. 5183–5189.

[9] S. Zhou, M. K. Helwa, and A. P. Schoellig, "Design of deep neural networks as add-on blocks for improving impromptu trajectory tracking," in *Decision and Control (CDC), 2017 IEEE 56th Annual Conference on*. IEEE, 2017, pp. 5201–5207.

[10] C. Sánchez-Sánchez and D. Izzo, "Real-time optimal control via deep neural networks: study on landing problems," *Journal of Guidance, Control, and Dynamics*, vol. 41, no. 5, pp. 1122–1135, 2018.

[11] S. Balakrishnan and R. Weil, "Neurocontrol: A literature survey," *Mathematical and Computer Modelling*, vol. 23, no. 1-2, pp. 101–117, 1996.

[12] M. T. Frye and R. S. Provence, "Direct inverse control using an artificial neural network for the autonomous hover of a helicopter," in *Systems, Man and Cybernetics (SMC), 2014 IEEE International Conference on*. IEEE, 2014, pp. 4121–4122.

[13] H. Suprijono and B. Kusumoputro, "Direct inverse control based on neural network for unmanned small helicopter attitude and altitude control," *Journal of Telecommunication, Electronic and Computer Engineering (JTEC)*, vol. 9, no. 2-2, pp. 99–102, 2017.

[14] F. Berkenkamp, M. Turchetta, A. P. Schoellig, and A. Krause, "Safe model-based reinforcement learning with stability guarantees," in *Proc. of Neural Information Processing Systems (NIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1705.08551

[15] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," *arXiv preprint arXiv:1802.05957*, 2018.

[16] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.

[17] T. Salimans and D. P. Kingma, "Weight normalization: A simple reparameterization to accelerate training of deep neural networks," in *Advances in Neural Information Processing Systems*, 2016, pp. 901–909.

[18] P. L. Bartlett, D. J. Foster, and M. J. Telgarsky, "Spectrally-normalized margin bounds for neural networks," in *Advances in Neural Information Processing Systems*, 2017, pp. 6240–6249.

[19] J. Slotine and W. Li, *Applied Nonlinear Control*. Prentice Hall, 1991.

[20] S. Bandyopadhyay, S.-J. Chung, and F. Y. Hadaegh, "Nonlinear attitude control of spacecraft with a large captured object," *Journal of Guidance, Control, and Dynamics*, vol. 39, no. 4, pp. 754–769, 2016.

[21] X. Shi, K. Kim, S. Rahili, and S.-J. Chung, "Nonlinear control of autonomous flying cars with wings and distributed electric propulsion," in *2018 IEEE Conference on Decision and Control (CDC)*. IEEE, 2018, pp. 5326–5333.

[22] D. Morgan, G. P. Subramanian, S.-J. Chung, and F. Y. Hadaegh, "Swarm assignment and trajectory optimization using variable-swarm, distributed auction assignment and sequential convex programming," *Int. J. Robotics Research*, vol. 35, no. 10, pp. 1261–1285, 2016.

[23] H. Khalil, *Nonlinear Systems*, ser. Pearson Education. Prentice Hall, 2002.

[24] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning requires rethinking generalization," *arXiv preprint arXiv:1611.03530*, 2016.

[25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[26] B. Neyshabur, S. Bhojanapalli, D. McAllester, and N. Srebro, "A pac-bayesian approach to spectrally-normalized margin bounds for neural networks," *arXiv preprint arXiv:1707.09564*, 2017.

[27] G. K. Dziugaite and D. M. Roy, "Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data," *arXiv preprint arXiv:1703.11008*, 2017.

[28] B. Neyshabur, S. Bhojanapalli, D. McAllester, and N. Srebro, "Exploring generalization in deep learning," in *Advances in Neural Information Processing Systems*, 2017, pp. 5947–5956.

[29] S.-J. Chung, S. Bandyopadhyay, I. Chang, and F. Y. Hadaegh, "Phase synchronization control of complex networks of Lagrangian systems on adaptive digraphs," *Automatica*, vol. 49, no. 5, pp. 1148–1161, 2013.

[30] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in pytorch," 2017.

Authorized licensed use limited to: CINVESTAV. Downloaded on September 11,2026 at 19:56:50 UTC from IEEE Xplore. Restrictions apply.